Implementation of Direction-Based Adaptive Beamforming and Spectral Post-Filtering for Noise Suppression on FPGA

Anonymous CVPR submission

Paper ID 10

Abstract

Spatial interference in communication systems results in loss of transmitted information. Beamforming addresses this by using a sensor array to filter out interference from unwanted directions, based on constructive and destructive combinations of signals. This project aims to implement hardware-feasible direction-based adaptive beamforming, followed by spectral post-filtering to suppress residual noise from the beamformed output, targeted for FPGA implementation. Beamforming approaches considered include MVDR, LCMV, and the Generalized Sidelobe Canceller, while noise-reduction approaches considered include Spectral Subtraction, Spectral Gating, Wiener Filtering, and MMSE/Log-MMSE estimation. This report reviews these methods, and they will be compared based on their noise-suppression performance, effect on signal quality, and computational cost, to guide the choice of methods best suited for hardware implementation. The selected design will first be built on the FPGA using recorded audio files, before moving to live microphones and speakers.

1. INTRODUCTION

A single receiver cannot distinguish between signals arriving from different spatial directions, so the received signal is corrupted by directional interference and noise. A uniform linear array of sensors overcomes this by utilizing the phase difference of an incoming wavefront among the different sensor elements. Combining signals from a desired direction constructively while suppressing others is the basic principle underlying beamforming.

Beamforming techniques are broadly grouped into classical and adaptive methods. Classical beamforming applies a fixed set of weights, determined by array geometry, in order to cause constructive interference of signals coming from a known direction. Common methods include Delay-and-Sum, Weighted Delay-and-Sum.

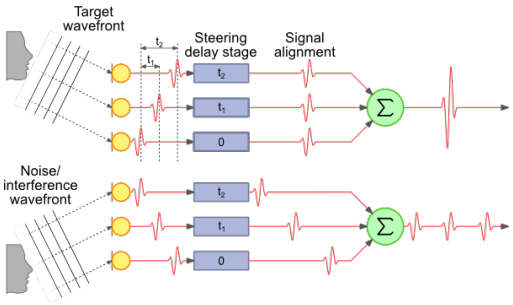


Figure 1. A weighted delay and sum beamformer. Source

In contrast to classical beamforming, which does not adapt weights according to changing signal environments, adaptive beamforming updates array weights based on the received signal statistics, allowing the array to track interference and noise conditions that change over time. This allows adaptive beamforming techniques to achieve effective spatial noise cancellation. MVDR (Capon), LCMV, and the Generalized Sidelobe Canceller (GSC) fall under this category.

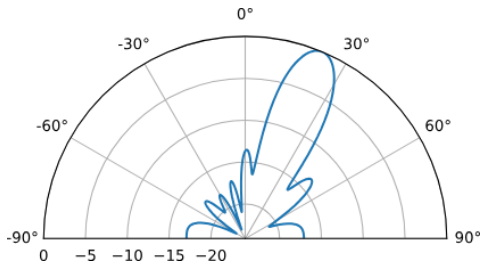


Figure 2. Beam pattern of a linear array in polar form. The angular axis is the direction of arrival and radial axis is the normalized array gain (dB) relative to the peak. The largest main lobe points in the look-direction while side lobes represent signals leaking in. Dips between lobes are spatial nulls. Source

The above mentioned adaptive beamforming methods are effective against directional interference, such as noise or interfering sources arriving from a specific angle relative

to the array, but are not designed to address diffuse, non-directional noise such as background hiss or ambient room noise with no single point of origin. These beamformers are correspondingly reported to give only limited SNR improvement against diffuse noise, with gains scaling with the number of microphones rather than the deep nulling achievable against a coherent point source [18]. Cheng *et al.* [6] explain this by pointing out that at low frequencies, the array cannot focus as sharply, and the noise picked up by different microphones starts to look nearly identical, leaving these beamformers with little spatial difference to work with in order to tell noise apart from the desired signal. In their tests, beamforming alone gave only a 0.33 dB improvement in signal to noise ratio, barely noticeable, while adding a post-filtering stage afterward pushed that improvement up to 6.90 to 9.41 dB, roughly 20 to 28 times better.

Once the beamformer isolates the desired signal from directional interference, a single-channel residual signal remains, still corrupted by non-directional ambient noise. Spectral post-filtering addresses this residual signal by utilizing differences between signal and noise statistics in the frequency domain. Noise suppression methods estimate the noise spectrum, typically during intervals where the desired signal is largely absent, and suppress it from the observed spectrum. Spectral Subtraction, Spectral Gating, Wiener Filtering, and MMSE/Log-MMSE estimation fall under this approach.

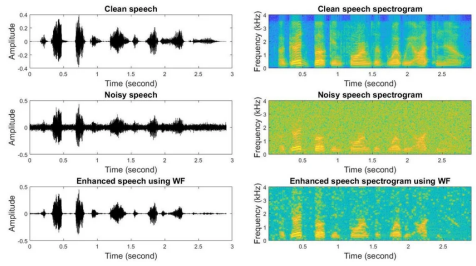


Figure 3. Noise reduction using Wiener Filter. Source

Beyond the algorithmic behavior of these techniques, real-time implementation introduces constraints absent from purely software treatments, such as finite arithmetic precision, limited hardware resources, and strict latency budgets. These constraints shape which methods are practically realisable on hardware. This project addresses direction-based adaptive multichannel beamforming of narrowband signals for the suppression of directional interference, followed by reduction of residual ambient noise using spectral post-filtering, targeted for FPGA implementation.

2. LITERATURE REVIEW

2.1. Adaptive Beamforming Methods

2.1.1 LCMV and MVDR

One of the foundational approaches to adaptive beamforming was presented by Frost [12], who formulated the problem as a linearly constrained optimization that enforces a desired response in a specified look direction and frequency band. It does not require any prior knowledge of the input-signal statistics, unlike several earlier approaches. In the original formulation each sensor is followed by a tapped delay line, enabling the processing of broadband signals. However, for the narrowband case considered in our project, they are omitted and each sensor is represented by a single complex spatial weight. Hence, the resulting Linearly Constrained Minimum Variance (LCMV) problem is defined as

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \mathbf{w}^H \mathbf{R}_{xx} \mathbf{w} \\ \text{subject to} \quad & \mathbf{C}^H \mathbf{w} = \mathbf{f}, \end{aligned} \tag{1}$$

where $\mathbf{w}$ is the vector of array weights, $\mathbf{R}_{xx}$ is the spatial covariance matrix of the received signals, $\mathbf{C}$ is the constraint matrix, and $\mathbf{f}$ contains the desired responses corresponding to each constraint.
For example, if θ_s denotes the direction of the desired signal and θ_i denotes the direction of a primary interferer, the beamformer can be constrained to have unit response in the desired direction and zero response in the interference direction. This can be expressed as

$$\mathbf{C}^H \mathbf{w} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \tag{2}$$

where

$$\mathbf{C} = \begin{bmatrix} \mathbf{a}(\theta_s) & \mathbf{a}(\theta_i) \end{bmatrix}. \tag{3}$$

The general optimization problem in Eq. (1) is derived using the principle that if the desired signal is held to a fixed response, minimizing the total output power is equivalent to minimizing the output power of noise alone. An important assumption made is that the non-look direction noise is uncorrelated to the signal.
The covariance matrix is generally unknown a priori and is therefore estimated from received snapshots. The estimated covariance matrix is then used to compute the beamforming weights.

A special case of Eq. (1) is the Minimum Variance Distortionless Response (MVDR) beamformer, in which the constraint set reduces to a single unity-gain constraint in the look direction θ_s:

$$\begin{aligned} \underset{\mathbf{w}}{\text{minimize}} \quad & \mathbf{w}^H \mathbf{R}_{xx} \mathbf{w} \\ \text{subject to} \quad & \mathbf{a}^H(\theta_s) \mathbf{w} = 1, \end{aligned} \tag{4}$$

which yields the closed-form solution

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{R}_{xx}^{-1}\,\mathbf{a}(\theta_s)}{\mathbf{a}^H(\theta_s)\,\mathbf{R}_{xx}^{-1}\,\mathbf{a}(\theta_s)}. \tag{5}$$

A limitation of Frost's LCMV beamformer is its sensitivity to small errors in the physical sensor array, such as sensor position, phase, or gain mismatches. To tackle this, Cox *et al*. [8] presented an improved recursive algorithm which includes multiple linear equality constraints and a quadratic inequality constraint. The uncorrelated errors in the sensors were modeled as white noise.

White noise gain (WNG) measures the gain of the array against uncorrelated spatially white noise and is used as a measure of the robustness to physical imperfections in the array [8]. It is defined as:

$$G_{\text{WNG}} = \frac{\left|\mathbf{w}^H\mathbf{a}(\theta_s)\right|^2}{\mathbf{w}^H\mathbf{w}}. \tag{6}$$

Under the unit-response constraint $\mathbf{a}^H(\theta_s)\mathbf{w} = 1$, this becomes

$$G_{\text{WNG}} = \frac{1}{\mathbf{w}^H\mathbf{w}}. \tag{7}$$

Therefore, $G_{\text{WNG}} \geq \delta^2$ is equivalent to the quadratic inequality constraint

$$\mathbf{w}^H\mathbf{w} \leq \frac{1}{\delta^2}. \tag{8}$$

As seen above, the array's sensitivity to uncorrelated noise decreases as WNG increases.

Two steps that dominate the hardware implementation cost of the above mentioned adaptive beamforming methods are forming the covariance matrix $\mathbf{R}_{xx}$ from the received snapshots, and inverting it to obtain the weights in Eq. (5). Direct inversion is $O(N^3)$ and involves division and square-root operations that are expensive and numerically sensitive in fixed-point arithmetic.

To address the cost of covariance matrix formulation and inversion in hardware Wang *et al*. [22] proposed a method with computational complexity of the order $O(N^2)$.The covariance matrix is first approximated by a Toeplitz matrix, exploiting the fact that, for a uniformly spaced array, the spatial correlation is approximately determined by the separation between sensors. The resulting structured matrix can then be decomposed efficiently using Cholesky decomposition with the Schur algorithm, reducing the computational complexity of the matrix operations. Though this work targets large arrays and an efficient high-throughput implementation, its treatment of low complexity operations is directly relevant to this project.

A parameterized, fixed point FPGA architecture was presented by Llamocca and Aloi [17], for a custom implementation of Frost's linearly constrained adaptive beamformer. The method presented implements complex weighting, accumulation and constraint update operations. Distributed arithmetic is used to reduce resource usage. The paper shows that optimally chosen fixed point representation can yield high accuracy while reducing resource dependence.

This provides a useful hardware-design reference for our implementation, particularly for selecting fixed-point precision and validating the beamformed output against a floating-point reference.

Chen *et al*. [5] note that conventional matrix inversion methods, such as Gauss–Jordan, QR, and LU decomposition, have high computational costs. The diagonal elements of the sample covariance matrix are presented to be substantially larger in absolute value than the off-diagonal elements. Hence, they play a larger role in determining the eigen-vectors and eigen-values, which govern the beamforming weights. Therefore, as a suitable approximation, off-diagonal elements are discarded, reducing the inversion in Eq. (5) to M independent reciprocal operations. Though FPGA implementation was not covered in the paper, on hardware, these can be implemented as parallel reciprocal operations, possibly decreasing resource usage.

2.1.2 Generalized Sidelobe Canceller (GSC)

The Generalized Sidelobe Canceller was proposed by Griffiths and Jim [13] as a way of turning the constrained beamforming problem of MVDR/LCMV into an unconstrained one. The received signal at the array is modelled as

$$\mathbf{x}(n) = \mathbf{a}(\theta_s)\,s(n) + \mathbf{n}(n), \tag{9}$$

where $\mathbf{x}(n)$ is the vector of sensor signals, $\mathbf{a}(\theta_s)$ is the steering vector for the desired signal arriving from direction θ_s, $s(n)$ is the desired signal, and $\mathbf{n}(n)$ is interference and noise. The GSC splits the array output into two branches. A fixed, quiescent beamformer keeps the response toward the desired direction unchanged, giving an output

$$y_q(n) = \mathbf{w_q}^H\mathbf{x}(n), \tag{10}$$

where $\mathbf{w_q}$ is the beamforming weight vector. Alongside this, a blocking matrix $\mathbf{B}$ satisfying

$$\mathbf{B}^H\mathbf{a}(\theta_s) = \mathbf{0} \tag{11}$$

removes the desired signal and passes only interference and noise into an adaptive branch,

$$\mathbf{u}(n) = \mathbf{B}^H\mathbf{x}(n). \tag{12}$$

CVPR #10

An adaptive filter estimates this unwanted component,

$$y_a(n) = \mathbf{g}^H(n)\,\mathbf{u}(n), \tag{13}$$

which is subtracted from the fixed branch's output to give the final signal,

$$y(n) = y_q(n) - y_a(n). \tag{14}$$

Because this avoids the direct matrix inversion needed in an explicit LCMV/MVDR solution, the problem reduces to a standard adaptive-filtering task, using LMS, NLMS, or RLS, which mainly involves additions, subtractions, and multiplications and is therefore easier to map onto FPGA hardware than large matrix operations.

The main limitation of the basic GSC comes from the blocking matrix. If the assumed direction of arrival does not exactly match the true source direction, due to sensor-position errors, gain mismatch, or a moving source, the blocking matrix can no longer fully null the desired signal. Part of it then leaks into the adaptive branch, where the filter treats it as interference and cancels it, so the GSC ends up removing some of the signal it was meant to preserve. Hoshuyama *et al.* [15] studied this problem and proposed the Robust GSC (RGSC), combining two changes: coefficient-constrained adaptive filters in the blocking matrix, which let it adapt within a limited range to correct direction errors of up to about 20° without drifting onto an interferer, and norm-constrained adaptive filters in the canceller stage, which cap the filter's coefficient strength so leftover leakage cannot be cancelled too aggressively. The norm constraint is

$$\|\mathbf{g}(n)\| \le \gamma, \tag{15}$$

where γ is the maximum permitted coefficient norm. Their listening tests improved from 2.8/5 for the conventional GSC to 3.8/5 with RGSC, and the method achieved over 30 dB of interference suppression in anechoic conditions and about 19 dB of noise suppression in a real reverberant room, at roughly twice the multiplication count of a fixed beamformer [15]. More recently, Hong *et al.* [14] proposed a robust RLS-based GSC that runs a network of GSCs pointed at several candidate directions instead of one fixed assumption, combining them through spatial regularization, with the RLS part implemented using a QR-decomposition structure for hardware efficiency, showing that DOA-mismatch robustness for GSC is still being actively developed.

On the hardware side, Zhao *et al.* [24] implemented the GSC on an FPGA for an 8-sensor array. They replaced the usual fixed blocking matrix with a DCT-based one, which decorrelates the sensor signals and speeds up convergence. The DCT used is

$$X_k = \sum_{n=0}^{N-1} x_n \cos\left[\frac{\pi}{N}\left(n+\frac{1}{2}\right)k\right], \tag{16}$$

where N is the transform length, x_n is the input sequence, and X_k is the k-th DCT coefficient. They also removed the division step from the LMS weight update by quantizing the normalization factor into a small number of levels, so it could be computed using canonical signed digit (CSD) shifts and additions instead of a hardware divider. Their fixed-point Verilog design on a Xilinx Virtex-7 (XC7VX330T) used at least 21% less hardware and ran at least 26.85% faster than a conventional divider-based version, with similar convergence behavior [24].

Together, these works show that the GSC avoids the large matrix inversions required by MVDR/LCMV, but its accuracy depends on how well the blocking matrix matches the true source direction, a problem addressed through added adaptive constraints [15] or spatial regularization [14] at some extra computational cost.

2.2. Spectral Post-filtering Methods

2.2.1 Spectral Subtraction

Spectral subtraction, proposed by Boll [4] in 1979, is based on a simple observation that the noise and the speech in a recording add together, so if we know roughly how much noise is present at each frequency, we can remove that amount from the signal. If $s(n)$ is the clean speech and $d(n)$ is the background noise, the microphone signal is

$$x(n) = s(n) + d(n). \tag{17}$$

The signal is first divided into short frames of $N = 256$ samples (about 32 ms at 8 kHz), each overlapping the previous one by half. Every frame is multiplied by a Hanning window,

$$w(n) = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \le n \le N-1, \tag{18}$$

which fades the frame smoothly to zero at both ends. This avoids sharp edges that would spread energy into the wrong frequencies, and it lets the overlapping frames add back to the original signal at the end. An FFT of each windowed frame gives $X(k)$ at frequency bin k, which we split into a magnitude $|X(k)|$ and a phase $\theta(k)$.

The noise estimate can be obtained from an initial noise-only segment or from detected noise-dominant frames.. The magnitudes of M noise-only frames are averaged to give the expected noise magnitude at each frequency,

$$\mu(k) = \frac{1}{M}\sum_{i=0}^{M-1} |X_i(k)|, \tag{19}$$

where Boll used about one third of a second of noise-only signal at the start [4]. This estimate is then subtracted from the magnitude of every new frame,

$$|\hat{S}(k)| = |X(k)| - \mu(k). \tag{20}$$

Because $\mu(k)$ is only an average, the noise in a particular frame can be smaller than $\mu(k)$, which makes the result negative. A magnitude cannot be negative, so such values are set to zero (half-wave rectification),

$$|\hat{S}(k)| = \max\big(|X(k)| - \mu(k),\, 0\big). \tag{21}$$

The noisy phase is reused without change, since the ear is far less sensitive to phase errors than to magnitude errors. The cleaned spectrum is

$$\hat{S}(k) = |\hat{S}(k)|\, e^{j\theta(k)}, \tag{22}$$

and an inverse FFT followed by overlap-add of the frames gives the enhanced waveform.

The method is attractive because it works on the waveform itself as a preprocessing step. Earlier options were noise-cancelling microphones, which give little reduction above 1 kHz, or modifying each voice processor internally, which is costly, so Boll's approach lets systems placed after it stay unchanged [4]. It needs a single microphone and about as much computation as a fast convolution.

Its main weakness is the leftover noise after rectification. In each frame, some frequencies keep small random peaks and others are cut to zero, so the residual sounds like faint tones switching on and off about every 20 ms. This is known as musical noise [4]. Boll reduced it by replacing each value with the minimum over three neighbouring frames, at the cost of one frame of delay and extra memory. Berouti *et al.* [3] instead subtract slightly more than the estimated noise and never allow the result to fall below a small fraction of the noisy signal,

$$|\hat{S}(k)|^2 = \max\big(|X(k)|^2 - \alpha\,\mu(k)^2,\; \beta\,|X(k)|^2\big), \tag{23}$$

where $\alpha \geq 1$ is the over-subtraction factor and $0 < \beta \ll 1$ is the spectral floor. Both must be tuned, and an FPGA study found $\beta = 0.01$ worked best for its system [9]. Two further limits come from the assumptions behind Eq. (19). The noise must stay roughly steady, and if it changes faster than the estimate can be updated, the reduction becomes less effective and can remove speech where little noise is present. A speech activity detector is also needed to decide when the estimate may be updated [4].

Spectral subtraction has been implemented on hardware several times. Bahoura and Ezzaidi [1] built it on an FPGA using a high-level programming tool, showing that the FFT, magnitude, subtraction and inverse FFT blocks fit a standard FPGA design flow. In an FPGA-based speech recognition system [9], the hardware version reached 80.28% average correlation with clean speech, against 83.7% for the MATLAB model, so fixed-point arithmetic cost a small amount of accuracy. A multi-band, phase-aware version has also been realised on a Virtex-6 FPGA [20]. Boll's original design already suits hardware, because real-valued speech lets two frames share one FFT, halving storage, and no zero padding is required [4]. The algorithm needs no matrix inversion, and its only demanding blocks are the FFT and IFFT, for which vendor cores exist, and the magnitude and phase computation, which a CORDIC block can perform using only shifts and additions. The window values in Eq. (18) can be stored in a ROM. The speech/silence decision can be made by comparing against a precomputed threshold, avoiding a logarithm and a division. The main points to watch are the word length of the fixed-point FFT and the added memory and delay if the three-frame minimum is used.

In our system, the beamformer output is a single channel, which is the setting spectral subtraction was designed for, so it needs no knowledge of the array geometry or the direction of arrival. It adds little cost on top of the resource-heavy beamforming stage, and its FFT may be shared with a frequency-domain beamformer. It targets the diffuse background noise that beamforming leaves behind [6, 18], and it gives a low-cost baseline for measuring how much a post-filtering stage improves the output.

2.2.2 Spectral Gating

Spectral subtraction removes the same average amount of noise at every frequency and, as noted above, produces musical noise and assumes steady noise. Spectral gating instead decides, for each time-frequency point, whether to keep or attenuate it, like a noise gate applied per band. This idea was developed into a practical algorithm by Sainburg and Zorea [19] in the *Noisereduce* package which is a free Python library. It needs no training data, only an estimate of the noise.

The signal is first converted to a spectrogram $X(k, l)$ with a short-time Fourier transform, where k is the frequency bin and l is the frame. The level of each point is

$$L(k, l) = 20 \log_{10} |X(k, l)|. \tag{24}$$

The noise is measured from a noise-only clip, or from quiet parts of the recording. For each frequency, we find the mean level $\mu_d(k)$ and the standard deviation $\sigma_d(k)$ of the noise. The gate threshold is set a little above the average noise level,

$$T(k) = \mu_d(k) + \gamma\, \sigma_d(k), \tag{25}$$

where γ controls how strict the gate is.

Each point is then compared with the threshold. Points above it are kept, and points below it are reduced. This gives the output

$$\hat{S}(k, l) = G(k, l)\, X(k, l), \tag{26}$$

where $G(k, l)$ is 1 for points above the threshold and a smaller value for points below it. The noisy phase is kept, and an inverse transform gives the cleaned signal.

The threshold can be fixed for the whole recording (stationary mode) or updated over time (non-stationary mode) [19]. The second mode can follow noise that changes, which spectral subtraction cannot do. The main weaknesses are that a high threshold removes weak consonants, and that a sudden change in the gate can cause musical noise.

2.2.3 Wiener Filtering, MMSE-STSA and log-MMSE

Noisy speech is modeled as

$$y(t) = x(t) + d(t), \tag{27}$$

where $x(t)$ is the clean speech we want and $d(t)$ is additive noise. Speech is quasi-stationary over 20–40 ms, so the signal is cut into short overlapping frames. Each frame is windowed and Fourier-transformed, giving one complex number per frequency bin k:

$$\begin{aligned} Y_k &= R_k\, e^{j\theta_k} \quad \text{(noisy)}, \\ X_k &= A_k\, e^{j\alpha_k} \quad \text{(clean speech)}, \end{aligned} \tag{28}$$

and D_k for the noise. R_k and A_k are amplitudes (how much energy is at frequency k), and θ_k and α_k are phases (timing information). The goal of every method here is to recover the clean component from the noisy Y_k, which is the only thing we can observe. Each method does this by multiplying Y_k by a gain between 0 and 1: a gain near 1 keeps a bin, and a gain near 0 suppresses it. The methods differ only in how they compute that gain.

Speech and noise are modeled as independent, zero-mean Gaussian random variables at each frequency [10]. The justification is the central limit theorem: each Fourier coefficient is a weighted sum of many samples, and sums of many small random contributions tend toward a Gaussian. For the amplitude a_k, this gives

$$p(a_k, \alpha_k) = \frac{a_k}{\pi\, \lambda(k)} \exp\!\left(-\frac{a_k^2}{\lambda(k)}\right), \tag{29}$$

where $\lambda(k) = \lambda_x(k)$ for speech and $\lambda(k) = \lambda_d(k)$ for noise, with

$$\lambda_x(k) = E\{|X_k|^2\}, \quad \lambda_d(k) = E\{|D_k|^2\} \tag{30}$$

being the average speech power and average noise power at bin k. Under this model, one number per bin fully describes the statistics of speech, and one describes the noise. Small amplitudes are common and large spikes are rare, with λ setting the scale. The phase does not appear in the formula because it is assumed uniformly random. Two SNR terms are used throughout: the a priori SNR, which is the expected long-term SNR of bin k, and the a posteriori SNR, which is how strong bin k looks in the current frame,

$$\xi_k = \frac{\lambda_x(k)}{\lambda_d(k)}, \qquad \gamma_k = \frac{R_k^2}{\lambda_d(k)}. \tag{31}$$

The Wiener filter [23] looks for the linear estimate $\hat{X}_k = G\, Y_k$ that minimizes the mean-square error on the whole complex value,

$$\min_G \; E\{\, |X_k - \hat{X}_k|^2 \,\}. \tag{32}$$

$$\hat{X}_k = G_w(\xi_k)\, Y_k, \qquad G_w(\xi_k) = \frac{\xi_k}{1+\xi_k}. \tag{33}$$

The gain behaves sensibly at both extremes: for $\xi_k = 9$ (clean bin), $G_w = 0.9$ and the bin is almost fully kept; for $\xi_k = 1$ (speech and noise equal), $G_w = 0.5$; for $\xi_k = 0.1$ (noise-dominated), $G_w \approx 0.09$ and the bin is heavily suppressed. The gain is real-valued, so it does not change the phase. The enhanced component keeps the noisy phase θ_k, and the amplitude estimate is

$$\hat{A}_k = G_w(\xi_k)\, R_k. \tag{34}$$

Two properties of this filter matter for what follows. First, the gain depends only on ξ_k, the long-term expected SNR, and ignores γ_k, so it does not react to how the current frame actually looks. Second, it was optimized for the complex value, so the amplitude estimate is only the modulus of that result, not something optimized for amplitude.

Hearing depends far more on the short-time spectral amplitude (STSA) than on the phase. Since the Wiener amplitude is only a byproduct, the MMSE-STSA estimator of Ephraim and Malah [10] minimizes the amplitude error directly,

$$\min \; E\{(A_k - \hat{A}_k)^2\} \;\Rightarrow\; \hat{A}_k = E\{A_k \mid Y_k\}. \tag{35}$$

The best estimate is the average clean amplitude given the observation. Evaluating this integral with the Gaussian model above gives $\hat{A}_k = G_{\mathrm{MMSE}}(\xi_k, \gamma_k)\, R_k$, where

$$\begin{aligned} G_{\mathrm{MMSE}} = \Gamma(1.5)\, \frac{\sqrt{v_k}}{\gamma_k}\, e^{-v_k/2} \\ \times \left[(1+v_k)\, I_0\!\left(\tfrac{v_k}{2}\right) + v_k\, I_1\!\left(\tfrac{v_k}{2}\right)\right], \end{aligned} \tag{36}$$

$$v_k = \frac{\xi_k}{1+\xi_k}\, \gamma_k. \tag{37}$$

Γ is the gamma function, and I_0 and I_1 are modified Bessel functions. Unlike Wiener, this gain depends on both ξ_k (prior expectation) and γ_k (current evidence). At high SNR ($\xi_k \gg 1$) it reduces to $\xi_k/(1+\xi_k)$, the Wiener gain, so the two agree when noise is low and differ at low SNR. The paper also shows that reusing the noisy phase is the optimal phase choice, so only the amplitude needs to be enhanced. Speech is not present in every bin of every frame, so [10]

adds q_k, the probability that speech is absent, and multiplies the gain by a confidence factor,

$$\hat{A}_k = \frac{\Lambda}{1+\Lambda}\, G_{\mathrm{MMSE}}(\xi_k, \gamma_k)\, R_k, \qquad \Lambda = \mu_k \frac{e^{v_k}}{1+\xi_k}, \quad \mu_k = \frac{1-q_k}{q_k}. \tag{38}$$

The factor is close to 1 when speech is likely present and close to 0 when it is likely absent.

The log-MMSE estimator (Ephraim and Malah, 1985) [11] keeps the same model but changes the error measure to the log of the amplitude, because loudness perception is roughly logarithmic,

$$\min\ E\{(\log A_k - \log \hat{A}_k)^2\} \quad \Rightarrow\ \hat{A}_k = \exp\big(E\{\ln A_k \mid Y_k\}\big). \tag{39}$$

Written out,

$$\hat{A}_k = \frac{\xi_k}{1+\xi_k}\, \exp\left\{\frac{1}{2}\int_{v_k}^{\infty} \frac{e^{-t}}{t}\, dt\right\} R_k = G_{\mathrm{logMMSE}}(\xi_k, \gamma_k)\, R_k. \tag{40}$$

The integral is the exponential integral, which takes the place of the Bessel functions. It is evaluated by differentiating a moment-generating function at $\mu = 0$. The gain again depends on ξ_k and γ_k, and Jensen's inequality shows it is never larger than the MMSE-STSA gain [11]. The authors did not combine it with the speech-absence extension, because it introduced a low-pass filtering effect on the speech [11].

The true ξ_k is unknown, so it is estimated with the decision-directed approach of [10], with $\alpha \approx 0.98$,

$$\hat{\xi}_k(n) = \alpha\, \frac{\hat{A}_k^2(n-1)}{\lambda_d(k, n-1)} + (1-\alpha)\, P\big[\gamma_k(n) - 1\big], \tag{41}$$

where $P[x] = \max(x, 0)$. The first term reuses the previous frame's cleaned amplitude, and the second uses the current frame's instantaneous SNR. A large α favors the smoothed history over noisy frame-to-frame fluctuations. In [10] the noise power $\lambda_d(k)$ was estimated once from an initial 320 ms noise-only segment, which assumes stationary noise. Both families of estimators therefore depend on accurate estimates of ξ_k and the noise power.

The Wiener gain is a single ratio and is the cheapest of the three to compute [23]. MMSE-STSA uses both ξ_k and γ_k, while Wiener uses only ξ_k, so the two agree at high SNR but MMSE-STSA does better at low SNR, which is exactly where noise suppression matters most [10]. It also has a guaranteed error bound, unlike Wiener, since it was directly built to minimize amplitude error [10]. In practice, listening tests showed MMSE with decision-directed SNR leaves smooth, steady residual noise, unlike the "musical noise" from spectral subtraction [10]. Log-MMSE improves on this further: its gain is proven to always be lower than MMSE-STSA's, which explains its even quieter residual noise without harming the speech itself [11].

For the system in [10], the processing pipeline is: first the signal is framed (256 samples, 192-sample overlap, 8 kHz), then a Hanning window is applied, then the FFT is taken, then ξ_k and γ_k are computed, then the gain is calculated, then the amplitude is multiplied by the gain, then the noisy phase is reattached, then the IFFT is taken, and finally the frames are combined by overlap-add. For MMSE-STSA, Ephraim and Malah used lookup tables of 961 gain samples in their system instead of evaluating the gain function directly. The tables covered the range -15 to $+15$ dB and were reported to add negligible residual noise for input SNRs of -5 to 5 dB [10]. The log-MMSE estimator was run in that same system [11]. The decision-directed estimate of ξ_k uses the previous frame's cleaned amplitude $\hat{A}_k$ [10].

In our system, the Wiener filter is the easiest option to start with. It needs just one division per frequency bin, so it is simple to build on an FPGA and fits well after the beamformer to clean up the noise. MMSE-STSA and log-MMSE work better when the noise is high, and they leave a smoother, less irritating noise behind. But they are harder to build, because they need a lookup table and they also need to remember the previous frame's output. Paper [21] found that MMSE can distort the speech more than Wiener, so the right choice depends on what we care about more: getting the amplitude as accurate as possible, or keeping the speech sounding natural.

3. NEXT STEPS

The next stage of the project is to compare the methods reviewed, choose one design, and build it on the FPGA. We plan to first work with recorded audio files, and then move to live microphones and speakers.

1. **Evaluation.** We will compare the different beamforming methods by how well they block the interfering source. We will compare the post-filtering methods by how much they improve the SNR and how much they distort the signal. We will also check how much better the output is with a post-filter than with beamforming alone.

2. **Hardware feasibility.** For each method, we will estimate how much FPGA logic and memory it needs, how much delay it adds, and whether it needs costly operations such as matrix inversion, division or lookup tables.

3. **Selection.** Using these results, we will choose one beamforming method and one post-filtering method

CVPR #10

that give a good balance between quality and hardware cost.

4. **FPGA implementation with audio files.** A noisy multichannel audio file will be given to the board as input, and the processed output will be read back and played or saved as an audio file. This lets us test the hardware without any analog front end.

5. **Live demonstration (future work).** If time permits, we will replace the audio file input with a real microphone array connected to the board, and send the output to a speaker.

4. EVALUATION OF ALGORITHMS

This section presents a software evaluation and comparison of the candidate algorithms reviewed in Sec. 2, for both the spatial filtering stage (adaptive beamforming, Sec. 4.1) and the single-channel post-filtering stage (noise suppression, Sec. 4.2). Its purpose is to establish the algorithmic choices and a floating-point baseline before the fixed-point hardware implementation. The two stages are evaluated separately; they are combined only in the system design of Sec. 3.

4.1. Adaptive Beamforming

For adaptive beamforming, the main algorithms considered in Sec. 2.1 were LCMV, MVDR and the GSC.

MVDR is a special case of LCMV with only one linear constraint, i.e. the distortionless response in the desired direction (Eq. (4)). Additional constraints, such as a null in the direction of a primary interferer, require the general LCMV formulation. However, in this problem statement the only input to the system is the desired direction of the signal, and interference directions are not specified. MVDR is therefore the member of the LCMV family that applies, and in the comparisons that follow, MVDR and GSC are compared. To keep the notation consistent with Sec. 2.1.1, the desired direction is written θ_s throughout.

4.1.1 Blocked MVDR Algorithm

The GSC (Sec. 2.1.2) updates its weights at every input sample. In contrast, MVDR requires an estimate of the spatial covariance matrix $\mathbf{R}_{xx}$ from the sensor data. Re-estimating this matrix and solving for the MVDR weights after every new sample is impractical: a single additional sample provides only limited information and may cause the covariance estimate and the weights to fluctuate, and continuously forming the covariance and inverting it adds substantial computational overhead.

Hence, instead of recomputing at every sample, the incoming sensor snapshots are grouped into blocks. The covariance matrix is estimated once per block, the MVDR weights are computed from it, and the weights are held fixed while that block is processed. We call this the "Blocked MVDR" approach. For block b containing L snapshots, the block covariance is

$$\widehat{\mathbf{R}}_b = \frac{1}{L}\sum_{n=1}^{L}\mathbf{x}[n]\mathbf{x}^H[n], \tag{42}$$

where $\mathbf{x}[n]$ is the vector of sensor samples and $(\cdot)^H$ denotes the conjugate transpose. The estimate used for the weights is updated recursively with forgetting factor λ,

$$\mathbf{R}_{xx,b} = \lambda\,\mathbf{R}_{xx,b-1} + (1-\lambda)\,\widehat{\mathbf{R}}_b, \tag{43}$$

and the weights follow from Eq. (5) with diagonal loading δ for numerical stability,

$$\mathbf{w}_{\mathrm{MVDR}} = \frac{(\mathbf{R}_{xx,b}+\delta\mathbf{I})^{-1}\mathbf{a}(\theta_s)}{\mathbf{a}^H(\theta_s)(\mathbf{R}_{xx,b}+\delta\mathbf{I})^{-1}\mathbf{a}(\theta_s)}. \tag{44}$$

The forgetting factor λ plays the same role for MVDR that the step size μ plays for the GSC, and is tuned below.

4.1.2 Comparison Methodology

The performance of the MVDR and GSC is evaluated using a common set of static and dynamic test cases. The comparison is carried out in two stages: parameter tuning followed by final evaluation.

Parameter tuning. Each algorithm has one setting that controls how quickly it reacts to changes in the signal environment. For the GSC this is the step size μ, which determines how fast its adaptive weights are updated: a small value gives a steady but slow response, while a large value reacts quickly but makes the output less stable. For the Blocked MVDR this is the forgetting factor λ of Eq. (43): a small value favours recent data and adapts quickly, while a large value averages over a longer history. All other settings, including the MVDR block length L and diagonal loading δ, were kept fixed.

Before the final comparison, six candidate values of μ (3×10^{-5} to 1×10^{-2}) and five candidate values of λ (0.5 to 0.98) were each run on every static and dynamic test case, using a small set of randomly generated signal and noise realizations that is separate from the one used for the final results. For each candidate, the resulting output SINR was averaged across the scenarios, with the static and dynamic groups weighted equally. The value with the highest average was selected for each algorithm ($\mu = 3\times10^{-4}$, $\lambda = 0.8$) and then kept unchanged for every scenario in the final comparison. Using one common setting, rather than

re-optimizing for each test case, ensures that neither algorithm is individually fitted to a particular scenario and gives a more consistent basis for comparison.

Final evaluation. With the settings fixed from the tuning stage, both algorithms are run on every test case. Each case is repeated for 20 independent random realizations, in which the noise samples and the starting phases of the signals are drawn separately. These realizations are different from those used during tuning, so the reported results are not tied to the particular noise on which the settings were chosen. The test scenarios themselves are the same as in tuning, and only the random realizations differ.

The test cases fall into two groups. In the static cases, the source directions and powers stay constant for the whole record, which allows the algorithms to be compared under steady interference. In the dynamic cases, the interference environment changes partway through the record: an interferer may appear, disappear, change power, move, or be replaced by one from a different direction. These cases test not only how well each algorithm suppresses interference but also how quickly it recovers when conditions change.

Performance is measured on the output of each algorithm. The first part of each record is excluded so that start-up behavior does not affect the result: the first half for static cases, and the first quarter for dynamic cases, which is before any of the scenario changes occur. For each test case, the SINR is averaged over the scored interval and then summarized across the 20 realizations by its mean and standard deviation. Finally, the difference between the two algorithms is computed within each realization, rather than from the averages, and the number of realizations in which MVDR outperforms GSC is counted. This gives a direct measure of which algorithm performs better under identical conditions.

4.1.3 Performance Evaluation Metrics

The algorithms are compared primarily using output SINR, with dynamic behavior assessed through the dynamic test suite.

- **Output SINR:** The signal-to-interference-plus-noise ratio at the beamformer output, in decibels, is

$$\mathrm{SINR} = 10 \log_{10} \left(\frac{P_s}{P_{i+n}} \right),$$

where P_s is the desired-signal power and P_{i+n} is the combined interference-plus-noise power. This is the primary metric, as it measures how effectively an algorithm suppresses interference while preserving the desired signal. Higher output SINR is better.

- **Output SNR:** The output signal-to-noise ratio, in decibels, is

$$\mathrm{SNR}_{\mathrm{out}} = 10 \log_{10} \left(\frac{P_s}{P_n} \right),$$

where P_n is the receiver-noise power at the beamformer output, excluding interference. This metric is secondary, because the beamformer output is subsequently processed by the post-filtering stage of Sec. 4.2, which is what reduces residual noise. It is therefore not tabulated here.

Two further criteria were considered but not made primary. The desired signal gain is always unity in the simulation, since the distortionless constraint of Eq. (4) is enforced by both algorithms. The white noise gain (Eq. (6)) measures robustness to sensor errors, which was not the focus of the software simulation, since the algorithms were evaluated under ideal array conditions.

4.1.4 Comparison Tables

Tab. 1 and Tab. 2 give the output SINR for the static and dynamic suites, with the comparative performance illustrated in Fig. 4. SINR is the main metric, since the main objective of beamforming is to remove directional interference.

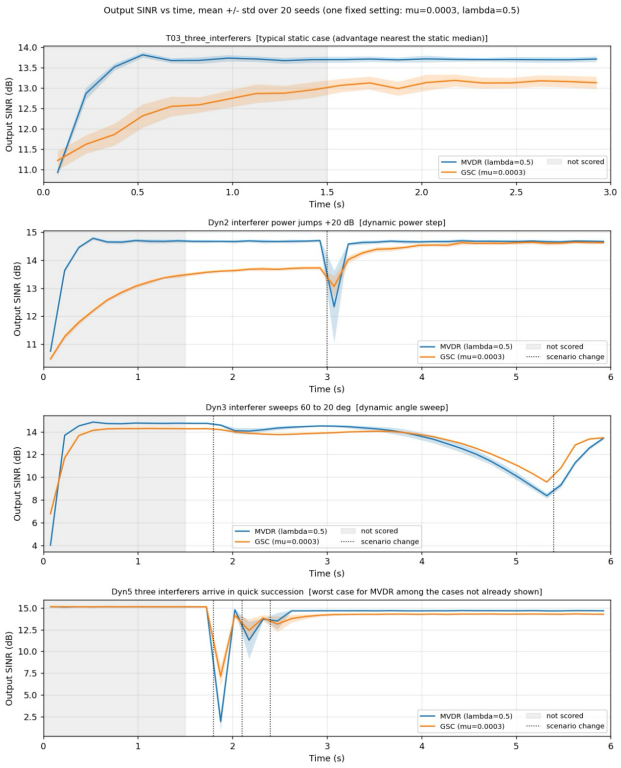


Figure 4. Comparison of output SINR between MVDR and GSC.

CVPR #10

Table 1. Static suite: steady-state output SINR (dB). Values are mean ± std across 20 seeds to 1 decimal place.

Case Description	Input SINR	MVDR	GSC	Δ (MVDR−GSC)	MVDR>GSC
No interference broadside	9.1	15.1 ± 0.0	15.1 ± 0.0	0.0 ± 0.0	20/20
Two separated	−3.3	14.7 ± 0.0	14.2 ± 0.0	0.5 ± 0.0	20/20
Three interferers	−5.0	13.7 ± 0.0	13.1 ± 0.0	0.6 ± 0.0	20/20
Four interferers	−6.2	11.4 ± 0.1	7.4 ± 0.4	4.0 ± 0.4	20/20
Six interferers	−7.9	8.2 ± 0.0	7.6 ± 0.1	0.7 ± 0.1	20/20
Near desired	−0.5	5.1 ± 0.0	4.9 ± 0.0	0.2 ± 0.0	20/20
Same direction	−0.5	−0.1± 0.0	−0.1± 0.0	0.0 ± 0.0	17/20
Duplicate directions	−5.0	14.8 ± 0.0	11.6 ± 0.0	3.2 ± 0.0	20/20
Angle boundary	−3.3	14.7 ± 0.0	13.1 ± 0.0	1.5 ± 0.0	20/20
Strong interferer	−20.0	14.1 ± 0.0	12.8 ± 0.0	1.4 ± 0.0	20/20
Weak low noise	17.0	27.9 ± 0.0	21.6 ± 0.0	6.3 ± 0.0	20/20
High noise	−4.8	5.6 ± 0.0	5.5 ± 0.0	0.1 ± 0.0	20/20
Near endfire	−3.3	13.5 ± 0.0	13.0 ± 0.0	0.4 ± 0.0	20/20
Symmetric interference	−3.3	14.9 ± 0.0	14.5 ± 0.0	0.4 ± 0.0	20/20
Mixed power interference	−8.7	11.5 ± 0.0	10.5 ± 0.4	1.0 ± 0.4	20/20
Mean over cases	−3.0	12.3 ± 0.0	11.0 ± 0.1	1.4 ± 0.1	–
Win / tie / loss for MVDR (±0.5 dB): MVDR wins 8, ties 7, GSC wins 0 (of 15 cases).					

Table 2. Dynamic suite: mean output SINR (dB) per scenario across 20 seeds.

Case Description	MVDR	GSC	Δ (MVDR−GSC)	MVDR>GSC
2nd intf. switches on	14.5 ± 0.1	14.3 ± 0.0	0.2 ± 0.1	19/20
Intf. power jumps +20 dB	14.6 ± 0.1	14.2 ± 0.0	0.4 ± 0.1	20/20
Intf. sweeps 60 to 20 deg	12.6 ± 0.0	12.9 ± 0.0	−0.4± 0.0	0/20
Intf. drops out, returns	13.0 ± 0.3	14.0 ± 0.0	−0.9± 0.3	0/20
Three intf. arrive quickly	12.5 ± 0.4	13.6 ± 0.2	−1.1± 0.3	0/20
Intf. hops 40 to −50 deg	12.7 ± 0.4	13.3 ± 0.1	−0.6± 0.4	2/20
Intf. sweeps 60 to 20, fast	11.3 ± 0.0	11.8 ± 0.0	−0.5± 0.0	0/20
Intf. approaches desired	4.2 ± 0.0	3.1 ± 0.0	1.1 ± 0.0	20/20
Mean over scenarios	11.9 ± 0.1	12.1 ± 0.0	−0.2± 0.1	–
Win / tie / loss for MVDR (±0.5 dB): MVDR wins 1, ties 3, GSC wins 4 (of 8 scenarios).				

4.1.5 Analysis of Simulation Results

The comparison results show that MVDR and GSC perform differently under static and changing interference conditions. In the static test cases, MVDR achieves a mean output SINR of 12.3 dB, compared with 11.0 dB for GSC. As seen in the comparison tables, MVDR clearly outperforms GSC in the static aspect.

The dynamic test cases give a slightly different result. MVDR achieves a mean output SINR of 11.9 dB, while GSC achieves 12.1 dB. MVDR wins one case, GSC wins four, and the remaining three are ties. GSC performs better in several cases where the interference changes rapidly, whereas MVDR performs well in some cases involving changes in interference power or direction. However, the difference in the performances of the two algorithms is not very large.

A plausible explanation for GSC's performance in the dynamic tests is that its weights are updated at the sample level, while the blocked MVDR implementation updates its weights once per block.

Selection:
Based on the analysis of the outputs, we have selected MVDR for further development because of its performance in the static tests and the relatively small difference between the two methods in the dynamic tests.

4.2. Noise Suppression (Post-Filtering)

For noise suppression, the five post-filters reviewed in Sec. 2.2 were compared: Spectral Subtraction (SS, Sec. 2.2.1), Spectral Gating (SG, Sec. 2.2.2), Wiener filtering, MMSE-STSA and log-MMSE (Sec. 2.2.3). All five act on the single-channel output of the beamformer, so they are compared on a common floating-point software model (`comparison.py`, available in our GitHub repository), following the design flow in [17]. The same model will later serve as the floating-point reference for the fixed-point FPGA design.

4.2.1 Common Processing Pipeline

All five methods share the same pipeline, so the only thing that changes between them is the gain rule $G(k,l) \in [0,1]$.

1. **STFT.** Hann window of 512 samples (32 ms at 16 kHz, the same frame duration as [10]), 75% overlap (hop 128), giving $Y(k,l)$ for bin k and frame l.

2. **Noise estimate.** The first $M = 20$ frames fall inside a silent lead-in and are assumed to be noise only:

$$\hat{\lambda}_d(k) = \frac{1}{M} \sum_{l=0}^{M-1} |Y(k,l)|^2. \tag{45}$$

   This is a mean *power*, computed once and kept fixed. It differs from the mean-*magnitude* estimator $\mu(k)$ of Eq. (19) used in the review of Spectral Subtraction (Sec. 2.2.1); both are standard choices, but they are not the same quantity.

3. **Gain.** Each method computes $G(k,l)$.

4. **Apply.** $\hat{S}(k,l) = G(k,l)\, Y(k,l)$, keeping the noisy phase, as in every method reviewed in Sec. 2.2.

5. **Inverse STFT** with overlap-add gives the cleaned signal.

4.2.2 Implemented Gain Rules

Wiener, MMSE-STSA and log-MMSE. These three share the same inputs. From Eq. (45), the a posteriori SNR is $\gamma_k(l) = |Y(k,l)|^2 / \hat{\lambda}_d(k)$, and the a priori SNR is the decision-directed estimate of Eq. (41) with $\alpha_{DD} = 0.98$:

$$\hat{\xi}_k(l) = \alpha_{DD} \frac{\hat{A}_k^2(l-1)}{\hat{\lambda}_d(k)} + (1-\alpha_{DD}) \max\big(\gamma_k(l) - 1, 0\big), \tag{46}$$

with $\hat{A}_k(l) = G(k,l)\,|Y(k,l)|$ and a floor of 10^{-6} on $\hat{\xi}_k$. With $v_k = \frac{\hat{\xi}_k}{1+\hat{\xi}_k}\gamma_k$:

$$G_W = \frac{\hat{\xi}_k}{1+\hat{\xi}_k}, \tag{47}$$

$$G_{\mathrm{MMSE}} = \frac{\sqrt{\pi v_k}}{2\gamma_k} \Big[(1+v_k)\, \tilde{I}_0\big(\tfrac{v_k}{2}\big) + v_k\, \tilde{I}_1\big(\tfrac{v_k}{2}\big) \Big], \tag{48}$$

$$G_{\log} = \frac{\hat{\xi}_k}{1+\hat{\xi}_k} \exp\Big(\tfrac{1}{2} E_1(v_k) \Big). \tag{49}$$

Here $\tilde{I}_n(x) = e^{-x} I_n(x)$ is the exponentially scaled Bessel function. This is the same formula as Eq. (36) (using $\Gamma(1.5) = \sqrt{\pi}/2$) with the $e^{-v_k/2}$ factor moved inside, which avoids numerical overflow. E_1 is the exponential integral of Eq. (40). Both MMSE gains are clipped to $[0,1]$. The speech-absence probability q_k of Eq. (38) is not implemented.

Spectral Subtraction. The over-subtraction form of Eq. (23), applied in the power domain with the noise power $\hat{\lambda}_d$ of Eq. (45):

$$G_{SS} = \sqrt{\frac{\max\big(|Y|^2 - \alpha_s \hat{\lambda}_d,\ \beta_s |Y|^2\big)}{|Y|^2}}. \tag{50}$$

The default values are $\alpha_s = 2$ and $\beta_s = 0.01$ (subscript s keeps them apart from α_{DD}). Boll's three-frame minimum [4] is not used.

Spectral Gating. The stationary, hard-gate threshold of [19] (Sec. 2.2.2), built per bin from the same M noise frames: $T(k) = \mu_d(k) + \kappa\, \sigma_d(k)$, where μ_d and σ_d are the mean and standard deviation of $20 \log_{10} |Y|$ over those frames:

$$G_{SG}(k,l) = \begin{cases} 1 & 20\log_{10}|Y(k,l)| > T(k), \\ g_f & \text{otherwise.} \end{cases} \tag{51}$$

The defaults are $\kappa = 1.5$ and $g_f = 0.1$. There is no smoothing over time or frequency.

4.2.3 Comparison Methodology

The comparison is carried out in two stages, parameter tuning followed by final evaluation, using the same set of test signals and noises for all methods.

Test signals and noise. All signals are sampled at $f_s = 16$ kHz, last 3 s, are scaled to a peak of 0.5, and are preceded by 0.3 s of silence, which is the noise-only segment used by Eq. (45) (the $M = 20$ frames span $512 + 19 \times 128 = 2944$ samples, which fits inside the $0.3 \times 16000 = 4800$ silent samples). Digitally generated signals are used because real speech was not available at this stage. Five signals (Tab. 3) are used. We call them *sparse-spectrum* rather than strictly narrowband, since a chirp or multi-tone covers a wide range

Table 3. Sparse-spectrum test signals.

Signal	Definition
Single tone	1000 Hz sine
Multi-tone	Sum of sines at 300, 1000, 2500, 4000 Hz
Chirp	Linear sweep from 200 to 6000 Hz over 3 s
AM tone	1500 Hz carrier, envelope $0.5\,(1+\sin 2\pi\,3t)$
Square wave	400 Hz square wave

of frequencies in total, although only a few bins carry energy in any single frame. This is consistent with the narrowband scope of the project stated in Sec. 1.

Three noise types are added at input SNRs of -5, 0, 5 and 10 dB, scaled so the SNR over the whole clip matches the target: white (flat spectrum), pink ($1/f$ roll-off, shaped from white noise), and narrowband (white noise through a 4th-order Butterworth band-pass, 1.1–1.3 kHz). The narrowband case overlaps the chirp's sweep and lies close to the square wave's third harmonic (1200 Hz), but mostly misses the single tone (1000 Hz) and the AM carrier (1500 Hz), so it is not equally hard for every signal. Since both the clean signal and the noise are generated here, the exact ground truth is known, which the metrics in Sec. 4.2.4 rely on.

Parameter tuning. SS and SG depend heavily on their parameters, so judging them only at default values would not be fair. We searched $\alpha_s \in \{2, 4, 6, 8, 12\}$, $\beta_s \in \{0.001, 0.01, 0.05\}$, $\kappa \in \{1.5, 2.5, 3.5, 5\}$ and $g_f \in \{0.005, 0.02, 0.05, 0.1\}$ using 2 tuning seeds (100, 101) kept separate from the evaluation seeds, and picked the setting with the best mean ΔSNR. The chosen values were $\alpha_s = 12$, $\beta_s = 0.001$, $\kappa = 3.5$ and $g_f = 0.005$, all at or next to the edge of the searched grid, so a wider search might do better still. Note that $\alpha_s = 12$ is much larger than the "slightly more than the noise" guidance of Sec. 2.2.1; the grid was chosen for SNR gain, not to match that guidance. The Wiener, MMSE-STSA and log-MMSE methods use $\alpha_{DD} = 0.98$ without tuning.

Final evaluation. Every combination of 5 signals, 3 noise types, 4 input SNRs and 5 evaluation seeds (0–4) is run for all five methods, giving $5 \times 3 \times 4 \times 5 \times 5 = 1500$ method-runs (300 runs per method, 60 conditions $\times$ 5 seeds). Reported values are averages over the full sweep. Differences between two methods are computed within each seed (paired comparison) and reported as mean $\pm$ standard deviation over seeds; a gap is treated as real only if it is clearly larger than this spread. A smaller supplementary run uses two noise-like signals (pink noise and band-limited noise used *as the signal*) to check whether a genuinely broadband signal changes the ranking.

4.2.4 Performance Evaluation Metrics

Because the test signals are not speech, speech-specific quality measures are not used. Instead, five signal-agnostic metrics are computed on the full clip, including the silent lead-in.

- **ΔSNR** (dB, higher is better): SNR of the cleaned signal minus SNR of the noisy one. This is the primary metric, since the purpose of the post-filter is to raise the SNR left by the beamformer.
- **Segmental SNR** (dB, higher is better): the same ratio on 20 ms frames, clipped to $[-10, 35]$ dB, averaged over active frames.
- **SI-SDR** (dB, higher is better): $\hat{s}$ is projected onto s and the ratio of projected to residual energy is taken in dB. This is the scale-invariant form widely used in source separation.
- **LSD** (dB, lower is better): RMS difference of the log-magnitude spectra, averaged over active frames. Both spectra are floored 60 dB below the clean spectrum's peak, since otherwise the empty bins of a sparse signal reach -200 dB and the value becomes meaningless.
- **ORMSE** (oracle-gain RMSE, lower is better): since the true signal S and noise D are known, the ideal gain $G_o = |S|^2/(|S|^2 + |D|^2)$ can be computed directly, and ORMSE is the RMS difference between a method's actual gain and G_o. As G_o has the exact shape of the Wiener gain (with the true instantaneous SNR in place of an estimate), this metric is naturally kind to Wiener-type gains and is secondary to the others.

RMSE and correlation are not reported, since RMSE is a fixed function of ΔSNR and correlation is a fixed function of SI-SDR, so neither can change a ranking. None of the metrics captures musical noise or perceived quality directly.

4.2.5 Comparison Tables

Tab. 4 gives the mean over all 300 runs per method at default parameters. Wiener gives the best value on all five metrics. Notably, MMSE-STSA has the worst ORMSE (0.462), worse than both SS (0.249) and SG (0.245), even though it is the more elaborate estimator.

At default settings SS and SG look much weaker than they really are. Tab. 5 and Fig. 5 show that tuning brings SS to 16.89 dB, only 0.4 dB below Wiener, and SG to 15.35 dB. Tuned SG also has a better LSD (5.55) than Wiener (6.60), so Wiener is best on all metrics only at default settings, not after tuning. The simple methods are therefore competitive

Table 4. Mean over all test runs, default parameters. ΔSNR, SegSNR, SI-SDR and LSD in dB; ORMSE is unitless.

Method	ΔSNR	SegSNR	SI-SDR	LSD	ORMSE
Spec. Subtraction	7.85	11.10	10.16	13.17	0.249
Spec. Gating	8.17	11.68	10.55	12.42	0.245
Wiener	**17.30**	**20.72**	**19.74**	**6.60**	**0.154**
MMSE-STSA	11.95	14.46	14.27	11.49	0.462
log-MMSE	14.50	17.18	16.85	9.52	0.207

Table 5. Default vs. tuned settings for the two simple methods (evaluation seeds).

Method	ΔSNR	SegSNR	SI-SDR	LSD	ORMSE
Wiener	17.30	20.72	19.74	6.60	0.154
SS, default	7.85	11.10	10.16	13.17	0.249
SS, tuned	16.89	19.67	19.67	6.80	0.168
SG, default	8.17	11.68	10.55	12.42	0.245
SG, tuned	15.35	18.60	17.56	5.55	0.196

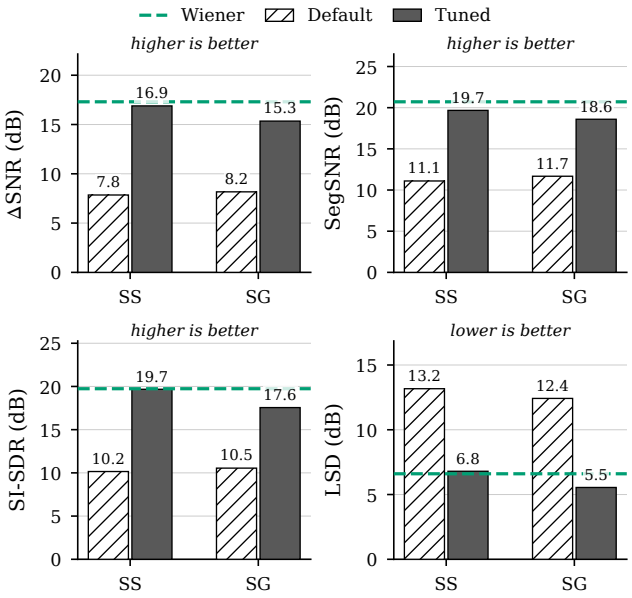


Figure 5. Performance metrics comparison between default and tuned configurations for Spectral Subtraction (SS) and Spectral Gating (SG). The dashed line represents the Wiener filter baseline.

once tuned, but they need that tuning, and aggressive subtraction or gating can cause musical noise that none of our metrics measures (Sec. 2.2.1, Sec. 2.2.2).

Tab. 6 and Fig. 6 give ΔSNR per signal. Wiener leads on four of the five signals, with a lead over MMSE-STSA of 4.4 to 8.2 dB. For the square wave, log-MMSE (9.04) is slightly ahead of Wiener (8.93).

Tab. 7 shows Wiener's ΔSNR advantage over the two MMSE estimators at each input SNR. It is largest at −5 dB

Table 6. ΔSNR (dB) per signal. SS and SG at default settings.

Signal	SS	SG	Wiener	MMSE	log-MMSE
Single tone	8.44	8.89	**21.19**	13.33	16.86
Multi-tone	7.93	8.23	**18.11**	12.17	14.78
Chirp	8.27	8.68	**16.65**	12.28	14.74
AM tone	8.49	8.94	**21.64**	13.44	17.06
Square wave	6.11	6.11	8.93	8.55	**9.04**

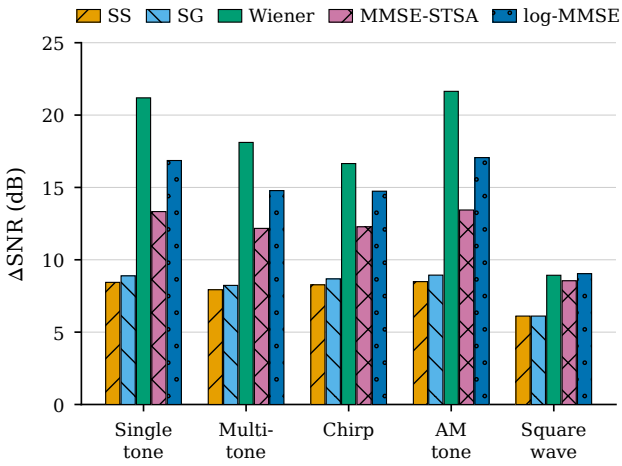


Figure 6. ΔSNR improvement across the five tested sparse-spectrum signals for all evaluated post-filtering methods.

Table 7. Wiener ΔSNR advantage (dB, mean ± std over seeds) by input SNR.

Input SNR	Wiener − MMSE-STSA	Wiener − log-MMSE
−5 dB	5.86 ± 0.11	3.06 ± 0.12
0 dB	5.57 ± 0.10	2.93 ± 0.13
5 dB	5.18 ± 0.10	2.74 ± 0.12
10 dB	4.79 ± 0.13	2.49 ± 0.08

and shrinks as the input gets cleaner. Sec. 2.2.3 reports that MMSE-type methods sound better at low SNR in listening tests; that concerns residual-noise quality, not SNR gain, so it is a different question from the one answered here rather than a contradiction. Wiener's advantage also holds for all three noise types, but by very different amounts: 3.67 ± 0.10 dB (white), 4.23 ± 0.64 dB (pink) and 8.15 ± 0.47 dB (narrowband).

4.2.6 Why Wiener Outperforms MMSE-STSA and log-MMSE

Although Sec. 2.2.3 describes the MMSE estimators as more advanced, three factors explain Wiener's superiority on our sparse signals:

1. Steeper fall-off in noise-only bins. For a small a priori SNR ($\xi \ll 1$), Wiener's gain falls linearly ($G_W \approx \xi$),

whereas the MMSE-family gains fall more slowly ($G \propto \sqrt{\xi}$). Fig. 7 illustrates this asymptotic difference. Consequently, MMSE estimators let significantly more noise through in empty bins.

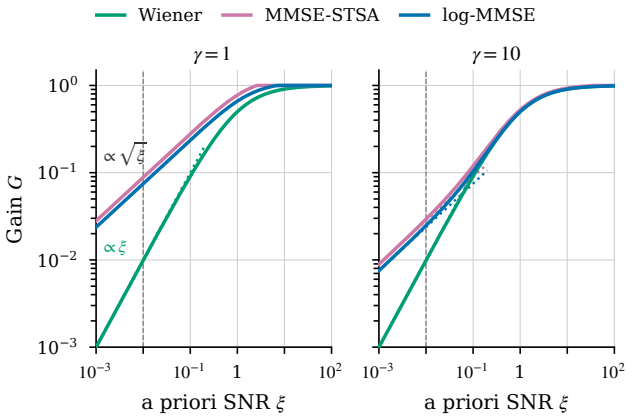


Figure 7. Theoretical gain curves $G(\xi, \gamma)$ versus a priori SNR (ξ) at fixed $\gamma = 1$ and $\gamma = 10$, demonstrating the steeper ξ falloff of the Wiener filter compared to the $\sqrt{\xi}$ falloff of the MMSE estimators.

2. High proportion of empty bins. Sparse signals like tones occupy only about 1% of a frame's bins. Tab. 8 shows that in the dominant signal-free bins, Wiener suppresses almost all noise (mean gain 0.011), while MMSE-STSA leaves roughly 39%. Although MMSE-STSA preserves the actual signal slightly better (0.85 vs. 0.72), SNR-based metrics heavily reward Wiener's aggressive noise suppression across the empty spectrum.

Table 8. Mean applied gain at 5 dB input SNR.

Method	Signal-free bins	Signal-dominated bins
Wiener	0.011	0.723
log-MMSE	0.085	0.741
MMSE-STSA	0.393	0.851

3. Statistical model mismatch. MMSE estimators assume the clean signal is a Gaussian random variable (Eq. (29)), fitting speech well but not steady, deterministic tones. They are also designed to prevent musical noise by keeping moderate gains in uncertain bins [10], prioritizing perceptual quality over raw SNR gain. Additionally, the decision-directed rule (Eq. (46)) feeds previous gains forward, meaning an initially aggressive filter like Wiener stays more aggressive.

Supporting evidence. The performance gap depends strongly on spectral sparsity. For the harmonic-rich square wave, Wiener's lead over MMSE-STSA nearly vanishes (0.4 dB, Tab. 6). In our supplementary run using broadband noise-like signals, Wiener's advantage disappeared completely, with log-MMSE pulling slightly ahead (-0.25 ± 0.13 dB). Thus, MMSE estimators are not inherently worse; they simply underperform when energy-based metrics evaluate highly sparse signals.

4.2.7 Analysis of Simulation Results

On this test set, the Wiener filter consistently yields the best ΔSNR. From a computational perspective, it is also the simplest of the three statistical-model methods: it requires only one division per bin and no lookup tables, whereas MMSE-STSA and log-MMSE require complex evaluations or table lookups for the Bessel functions and the exponential integral E_1. All three methods require storing the previous frame's output amplitude to estimate $\hat{\xi}_k$ (Eq. (46)). Furthermore, the division by the noise estimate $\hat{\lambda}_d(k)$ can be optimized as a multiplication by a precomputed reciprocal, since $\hat{\lambda}_d(k)$ remains fixed after the initialization phase.

Based on these simulation results, Wiener filtering provides the best balance of aggressive noise suppression and computational efficiency for sparse-spectrum signals. Additionally, tuned Spectral Subtraction performs closely in overall SNR gain and remains a viable, even lower-complexity alternative.

To reproduce these results: `pip install numpy scipy`, then run `python comparison.py`. The full run takes about 5 minutes (1 minute with `RUN_TUNING = False`) and writes the result tables as CSV files. Evaluation seeds are 0–4 and tuning seeds 100–101, so the numbers above reproduce exactly.

5. LDL-Based MVDR Implementation

5.1. Computational Requirements of Block-Adaptive MVDR

In the block-adaptive implementation, the input signal is processed in blocks of 1024 samples from the four antenna elements. For each block, the processor estimates how the signals are correlated across the antennas (the covariance matrix), smooths this estimate with that of earlier blocks so that it tracks changes over time, and adds a small stabilising term to its diagonal to improve numerical conditioning. The beamformer weights are then computed from this matrix and applied to the next block.

Most of these steps are inexpensive. Accumulating the covariance estimate and smoothing it consist of simple multiply-accumulate operations.
The weight calculation is different; it requires the effect of the inverse of the covariance matrix, which is a much heavier and less regular operation involving division, strict data dependencies between steps, and sensitivity to numerical precision. In practice, the full inverse is not formed explicitly. Instead, the weights are obtained by solving a small lin-

ear system and then normalising the result, which is cheaper and numerically safer.

This step is the main hardware bottleneck. Its cost grows quickly with the matrix size, it needs division and higher arithmetic precision than the rest of the design. These properties make it the most demanding part in terms of logic resources and latency.

Hence, to implement matrix inversion in a cost effective and stable way several decompositions and factorisations exist. They are discussed in the subsequent subsections.

5.2. Matrix Factorisation Methods

Two established methods for factorising a positive-definite covariance matrix (which is created in the MVDR algorithm) are Cholesky factorisation and LDL^H factorisation. Both can be used to obtain the MVDR weights, but their computational requirements differ.

Cholesky factorisation expresses the loaded covariance matrix as

$$\mathbf{R}_L = \mathbf{L}_c \mathbf{L}_c^H, \tag{52}$$

where $\mathbf{L}_c$ is a lower-triangular matrix with a real, positive diagonal. The diagonal elements involve square-root operations, while the remaining elements are calculated using multiplication, subtraction, and division.

LDL^H factorisation expresses the same matrix as

$$\mathbf{R}_L = \mathbf{L}\mathbf{D}\mathbf{L}^H, \tag{53}$$

where $\mathbf{L}$ is a unit lower-triangular matrix and $\mathbf{D}$ is a real diagonal matrix. Unlike conventional Cholesky factorisation, this form does not require square-root operations.

An equivalent upper-triangular form is

$$\mathbf{R}_L = \mathbf{U}^H \mathbf{D}\mathbf{U}, \tag{54}$$

where $\mathbf{U} = \mathbf{L}^H$ is unit upper triangular.

The absence of square roots makes LDL^H an attractive candidate for hardware implementation. However, this alone does not establish that it will always use fewer resources or achieve better timing than Cholesky. The result depends on the arithmetic representation, architecture, and synthesis tool.

5.2.1 Comparison of LDL-Based and Cholesky-Based Approaches

Cho *et al.* [7] compared LL^T and LDL^T implementations of a linear-system solver for 4×4 matrices. Their design used fixed-point arithmetic and square root operations were also performed for the Cholesky decomposition.

Their reported results are summarised in Table 9.

The authors report a reduction in hardware size for their LDL^T implementation. The table also shows that the LDL^T design uses more multiplications and slightly more DSP blocks. Its advantages therefore depend on the resources considered and on the implementation cost of the square-root operation. The advantage of the LDL implementation lies in the fact that it bypasses square root calculation, which is slightly intensive in hardware.

Table 9. Comparison reported by Cho *et al.* [7]. These are the authors' results, not results from the present implementation.

Parameter	LL^T	LDL^T
Slices	7636	5243
Slice flip-flops	12472	8780
4-input LUTs	6018	3814
DSP48	22	24
Maximum clock frequency (MHz)	31.320	58.210
Latency	7	5
Multiplications	22	32
Divisions / square roots	14 / 4	10 / 0

Ingemarsson and Gustafsson [16] investigated an LDL-based algorithm for inverting positive-definite matrices. Their approach performs LDL factorisation followed by direct back-substitution on the upper-triangular part, avoiding the need to calculate a separate triangular inverse or perform an additional matrix multiplication for that purpose. The method is designed for hardware implementation.

Their work is particularly relevant to this project because it describes a specific LDL-based matrix-inversion algorithm that can be adapted to the 4×4 Hermitian covariance matrix used by our four-antenna beamformer.

Based on the computational requirements of the problem and the approaches described in the literature, LDL^H was selected for further development. The next subsection describes how the selected algorithm relates to the reference implementation.

5.3. The Selected LDL-Based Algorithm and Its Relation to the Reference Paper

5.3.1 The Reference Algorithm

Ingemarsson and Gustafsson [16] describe a method for inverting a positive-definite matrix using LDL factorisation followed by direct back-substitution.

Using the notation adopted in this report, the factorisation is

$$\mathbf{A} = \mathbf{U}^H \mathbf{D}\mathbf{U}, \tag{55}$$

where $\mathbf{A}$ is Hermitian positive definite, $\mathbf{U}$ is unit upper triangular, and $\mathbf{D}$ is real diagonal.

The factorisation can be written in terms of the diagonal pivots d_i and the upper-triangular elements U_{ij}. For $j > i$,

$$d_i = A_{ii} - \sum_{k<i} |U_{ki}|^2 d_k, \tag{56}$$

and

$$U_{ij} = \frac{A_{ij} - \sum_{k<i} U_{ki}^{*} d_k U_{kj}}{d_i}. \quad (57)$$

Once the factors have been calculated, the inverse can be obtained using the triangular structure of the problem. The reference method avoids square-root operations and does not require explicitly forming a separate triangular inverse before the back-substitution stage.

The paper also presents a hardware architecture based on a triangular array of processing elements, modulo scheduling, and interleaving. These features allow multiple matrix operations to be overlapped in hardware. However, the architecture is distinct from the sequential implementation aimed to be developed in this project.

5.3.2 The Reference Architecture and Its Reported Results

The reference paper maps the matrix-inversion algorithm onto a triangular processing array containing $N(N-1)/2$ multiply-accumulate cells, together with a shared reciprocal unit. For a 4×4 matrix, this corresponds to ten cells.

The authors report an implementation for 4×4 and 8×8 complex matrices on an AMD/Xilinx Virtex-7 device using ISE 14.7. Their design uses an 18-bit numerical representation and a Newton–Raphson reciprocal unit.

For the 4×4 configuration, the paper reports 34 DSP48E1 blocks, 4231 LUTs, 4454 registers, a latency of 40 cycles, a maximum clock frequency of 160 MHz, and a throughput of 40 million matrix inversions per second.

These figures describe the reference paper's implementation on its own target device and architecture. They are included to explain the published design and are not reflections of our aim through the project.

Our implementation adopts the matrix-inversion algorithm explained in this paper but uses a sequential architecture with shared arithmetic operations. It does not reproduce the paper's systolic array, modulo schedule, or interleaving mechanism.

5.3.3 Comparison with Our C++ Implementation

The selected algorithm was implemented in C++ for AMD Vitis High-Level Synthesis. The functions `mvdr_rdr_invert` and `solve_mvdr` contain the matrix-inversion and MVDR weight-calculation stages, respectively.

The implementation uses the same general LDL-based matrix-inversion approach as the reference paper, but the arithmetic representation and hardware architecture differ.

The current code also includes diagonal loading, a pivot floor to protect against very small pivots (division of zero), and normalisation of the resulting weights to satisfy the MVDR distortionless-response constraint.

Thus, the implementation should be described as an adaptation of the reference paper's matrix-inversion algorithm rather than a reproduction of its complete hardware architecture or reported experimental results.

6. HARDWARE FEASIBILITY

6.1. HLS Synthesis Results of MVDR

The C++ implementation was synthesized using Vitis HLS 2025.2 for the AMD/Xilinx Spartan-7 XC7S50 FPGA. The reported results for the `mvdr_block_top` configuration are summarized in Table 10.

Table 10. HLS resource and timing estimates for the LDL-based MVDR implementation.

Parameter	Reported result
Target clock period	40.00 ns (25 MHz)
Estimated clock period	35.375 ns
Clock uncertainty	4.80 ns
Timing slack after uncertainty	−0.17 ns
Estimated latency	8,727–10,217 cycles
LUTs	11,117/32,600 (34%)
Flip-flops	12,410/65,200 (19%)
DSP blocks	55/120 (45%)
BRAM18K	0/150 (0%)

The synthesis results indicate that the design uses approximately 34% of the available LUTs, 19% of the flip-flops, and 45% of the DSP blocks reported for the target device. These estimates suggest that the current processing core fits within the device's reported logic and DSP resources, although additional resources will be required for buffering, control, and input/output handling in a complete system.

The estimated clock period is below the 40 ns target before accounting for clock uncertainty. However, the reported timing slack after uncertainty is −0.17 ns, indicating a small timing violation in the HLS estimate. Timing therefore remains an issue to address before considering the design ready for hardware operation.

The reported latency is substantially shorter than the 128 ms duration of an input block at 8 kHz. This is encouraging for the proposed block-based architecture, but the estimate alone does not establish real-time operation on the physical FPGA. The design must still be evaluated through RTL verification and Vivado implementation, including post-placement-and-routing timing analysis.

The reported BRAM usage is zero for this synthesized core. This does not mean that the complete beamforming system requires no memory. Input and output storage,

CVPR #10

buffering, and any additional logic required to stream or replay blocks must be considered when integrating the design on the board.

6.2. HLS Synthesis Results of Wiener Filtering

This section explains the post-filter that was actually built for the FPGA, how it relates to the time-domain Wiener filter of Basiri [2] that it started from, and whether it fits the target device (Xilinx Spartan-7 XC7S50) in logic, memory, delay and arithmetic cost.

6.2.1 Reference Design: Time-Domain Wiener Filter

Basiri [2] implements the Wiener filter directly in the time domain by solving the Wiener–Hopf equation

$$\mathbf{H} = \mathbf{R}_{xx}^{-1}\,\mathbf{R}_{sx}, \tag{58}$$

where $\mathbf{R}_{xx}$ is the $n \times n$ autocorrelation matrix of the noisy input and $\mathbf{R}_{sx}$ the cross-correlation between the desired signal and the input. The inverse is obtained by a Givens-rotation QR decomposition, $\mathbf{R}_{xx} = \mathbf{Z}^T\mathbf{U}$, so that $\mathbf{R}_{xx}^{-1} = \mathbf{U}^{-1}\mathbf{Z}$. This needs $n(n-1)/2$ rotations, each built from a square root and a division, and inverting $\mathbf{U}$ through its adjoint costs about $n^2(n-2)$ multiplications and $n(n+1)/2$ divisions. The design runs on a Zynq-7000 device, where two ARM cores feed small reconfigurable multiply/add and square-root/divide co-processors over AXI-Lite, uses half-precision floating point with 16×16 matrices, and reports a delay of 0.41 s for one 16-point filtering operation.

This is a valid hardware Wiener filter, but it is not suited to our chain for three reasons. The matrix work grows as $O(n^3)$ and needs square roots and many divisions, which are the operations we want to avoid. The processors orchestrate every arithmetic step, so the delay is orders of magnitude above the real-time budget of an audio stream. And it computes one coefficient set per block, whereas the software model re-estimates the a priori SNR in every frame.

6.2.2 From Time Domain to Frequency Domain

For a stationary input, $\mathbf{R}_{xx}$ is Toeplitz, and for long frames it is well approximated by a circulant matrix, which the DFT diagonalises:

$$\mathbf{R}_{xx} \approx \mathbf{F}^H \operatorname{diag}\big(P_x(k)\big)\,\mathbf{F} \;\Rightarrow\; H(k) = \frac{P_{sx}(k)}{P_x(k)} = \frac{\xi_k}{1+\xi_k}. \tag{59}$$

Here $P_x = P_s + P_d$ and $P_{sx} = P_s$ for uncorrelated signal and noise, and $\xi_k = P_s/P_d$. The $n \times n$ inversion of Eq. (58) thus collapses into N independent scalar divisions, and the result is exactly the Wiener gain G_w of Eq. (33). Since the signal is only quasi-stationary, the filter is applied frame by frame (STFT) and ξ_k is re-estimated every frame with the decision-directed rule of Eq. (46). The price is two FFTs per frame, $O(N \log N)$, instead of an $O(n^3)$ matrix inversion. Windowing and overlap-add keep the circular-convolution approximation from creating frame-edge artefacts.

Table 11. Reference time-domain design vs. this work.

Aspect	Ref. [2]	This work
Domain	Time (Wiener–Hopf)	STFT, per-bin gain
Core operation	$\mathbf{R}_{xx}^{-1}$ by Givens QR	512-pt FFT / IFFT
Cost per update	$O(n^3)$	$O(N \log N)$
Special ops	square root, many divisions	one division per bin
Arithmetic	half-precision float	float32 inside, fixed-point I/O
Platform	Zynq-7000, ARM cores + co-processors	stand-alone streaming IP (Vitis HLS), XC7S50
Operation	batch, 16-point block	streaming, 128 new samples per call

6.2.3 Algorithm Implemented in Hardware

The hardware uses the same gain rule as the software Wiener model (Sec. 4.2.1); only the data flow is made causal. With frame length $N = 512$, hop 128 (75% overlap) and the periodic Hann window $w(n)$, one hop is processed as follows.

1. **Window and FFT.** The newest 512 samples are windowed and transformed, $Y_k(l) = \mathrm{FFT}\{w(n)\,x_l(n)\}$, $k = 0, \ldots, 256$.

2. **Noise learning.** After 3 start-up frames, the power $|Y_k|^2$ of the next $M = 20$ frames is accumulated. At the end the design stores the reciprocal $\rho_k = 1/(\hat{\lambda}_d(k) + \varepsilon)$, so the main loop needs no divider for the noise term. The output is muted during these $3 + 20 = 23$ hops.

3. **Gain.** With $P_k = |Y_k|^2$,

$$\hat{\xi}_k = \max\big(\alpha_{DD} A_{k,\mathrm{prev}}^2 \rho_k + (1-\alpha_{DD})\max(P_k\rho_k - 1,\, 0),\, 10^{-6}\big). \tag{60}$$

$$G_k = \frac{\hat{\xi}_k}{1+\hat{\xi}_k}, \qquad A_{k,\mathrm{prev}}^2 \leftarrow G_k^2 P_k. \tag{61}$$

   Storing $A^2 = G^2|Y|^2$ instead of $A = G|Y|$ removes the square root.

4. **Inverse FFT on the same core.** The gained spectrum $Z_k = G_k Y_k$ is completed by $Z_{N-k} = Z_k^*$ and transformed back with the *forward* FFT using $z = \frac{1}{N}\big(\mathrm{FFT}\{Z^*\}\big)^*$, so one FFT engine serves both directions.

5. **Overlap-add.** The result is multiplied by $w(n)$, added into a circular accumulator, and the oldest 128 completed samples are emitted after division by $\sum w^2 = 1.5$.

The streaming model differs from the Python model in three ways. First, it learns the noise PSD from frames 3–22 instead of frames 0–19. Second, the output is delayed by 384 samples (24 ms at 16 kHz). Third, the first three enhanced hops are only partially reconstructed, and full reconstruction starts from the fourth hop. Since the evaluation uses a 0.3-second lead-in (37 hops), noise learning finishes before the test signal begins. Therefore, these initial effects do not affect the test signals.

6.2.4 Architecture

One call of the top function processes 128 new samples and produces 128 output samples. All internal state, including input history, the overlap-add buffer, noise reciprocal, and previous A^2 values, is stored in circular buffers and on-chip memory. The same radix-2 FFT is used for both forward and inverse transforms, so only one FFT unit is needed. The design uses cosine and sine lookup tables with 256 entries each and stores the Hann window in ROM.

To save hardware area, the design uses only one floating-point multiplier, adder, subtracter, and divider, trading speed for smaller area. The input uses 18-bit fixed-point values with 5 integer bits, while the output uses 24-bit fixed-point values with 5 integer bits. Internal processing uses single-precision floating point to stay close to the software reference. Reducing the internal word length is left for future fixed-point testing.

Vitis HLS generates start and done signals to control the block. The 128-sample input and output buffers are exposed as memory ports, allowing a small buffer or DMA wrapper to connect to the block.

6.2.5 Resource and Latency Estimate

The block was synthesised with Vitis HLS 2025.2 for the XC7S50 with a 10 ns (100 MHz) clock target. The figures below are HLS estimates before place-and-route.

Real-time budget. At 16 kHz, one hop of 128 samples lasts 8 ms, i.e. 800 000 clock cycles at 100 MHz. The worst-case latency of one call is 35 869 cycles (0.359 ms), about 4.5% of the budget, so the block runs roughly 22 times faster than real time. Tab. 12 shows where the cycles go. The two FFT passes dominate (13 845 cycles each); with 2 304 butterflies per pass this is about 6 cycles per butterfly, slower than the 4 cycles targeted, which is still far inside the budget. The call is not pipelined across hops, so the throughput equals the latency. The shortest call, 402 cycles, occurs during the three start-up frames, which only store samples and emit zeros.

Table 12. Latency per call of the main stages (cycles, Vitis HLS report).

Stage	Cycles
Push 128 new samples	130
Window + bit-reversed load	1 031
Forward FFT	13 845
Wiener gain, 257 bins	4 160
Inverse FFT (same engine)	13 845
Synthesis window + overlap-add	2 062
Output 128 samples	266
Worst case per call (incl. control)	35 869
Only while learning the noise:	
Noise accumulation (per learning hop)	1 043
Noise reciprocals (once, 257 bins)	7 196

Resources. Tab. 13 shows that the design uses 20% of the LUTs, 10% of the DSP slices and 9% of the block RAM of the XC7S50, so it leaves most of the device free for the beamformer. The estimated clock period is 8.747 ns; with the 1.25 ns uncertainty margin this is 9.997 ns, so the 10 ns target is met with almost no slack and should be confirmed after place-and-route. The 14 block RAMs hold about 140 kbit: 124 kbit in nine arrays (the four 512-point work arrays, the overlap-add buffer, the input history, the noise-reciprocal and A^2 vectors, and the Hann ROM) and 16 kbit of twiddle ROM inside the FFT. The 12 DSP slices are the floating-point multiplier and adder cores of the FFT and of the gain logic.

Table 13. Synthesis estimate for the XC7S50 (Vitis HLS, 10 ns clock).

Resource	Used	Available	Use
LUT	6 837	32 600	20%
Flip-flops	4 729	65 200	7%
DSP48	12	120	10%
BRAM (18 kb)	14	150	9%
Clock period (est.)	8.747 ns + 1.25 ns unc.		
Worst-case latency	35 869 cycles (0.359 ms)		

Costly operations. There is no matrix inversion and no square root. The only divider is the single 16-cycle floating-point division $\xi/(1+\xi)$ per bin, plus 257 one-off reciprocals when learning ends (28 cycles each). No look-up table is needed for the gain, which is the main advantage over MMSE-STSA and log-MMSE (Bessel and E_1 evaluations). The only constant tables are the twiddle and window ROMs.

References

[1] M. Bahoura and H. Ezzaidi. Implementation of spectral subtraction method on fpga using high-level programming tool.

In *Proc. 24th Int. Conf. Microelectronics (ICM)*. IEEE, 2012. 5

[2] Mohamed Asan Basiri M. An efficient FPGA implementation of Wiener filter. In *2024 IEEE 8th International Conference on Information and Communication Technology (CICT)*. IEEE, 2024. 17

[3] M. Berouti, R. Schwartz, and J. Makhoul. Enhancement of speech corrupted by acoustic noise. In *Proc. IEEE ICASSP*, volume 4, pages 208–211, 1979. 5

[4] Steven F. Boll. Suppression of acoustic noise in speech using spectral subtraction. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 27(2):113–120, 1979. 4, 5, 11

[5] Weiheng Chen, Yihuan Zhao, and Jing Gao. Improved Capon beamforming algorithm by using inverse covariance matrix calculation. In *IET International Radar Conference 2013*, pages 1–6, 2013. 3

[6] Ning Cheng, Wen-Ju Liu, Peng Li, and Bo Xu. Microphone array speech enhancement based on a generalized post-filter and a novel perceptual filter. In *Proc. ICSP2008*, pages 370–373. IEEE, 2008. 2, 5

[7] Hanjoon Cho, Jinyong Lee, and Younglok Kim. Efficient implementation of linear system solution block using LDL^T factorization. In *2008 International SoC Design Conference (ISOCC)*, pages III–19–III–20. IEEE, 2008. 15

[8] Henry Cox, Robert M. Zeskind, and Mark H. Owen. Robust adaptive beamforming. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(10):1365–1376, 1987. 3

[9] De La Salle University. Spectral subtraction speech enhancement integrated to automatic speech recognition system implemented in fpga. Master's thesis, De La Salle University, 2012. 5

[10] Y. Ephraim and D. Malah. Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator. *IEEE Trans. Acoust., Speech, Signal Process.*, ASSP-32(6):1109–1121, Dec. 1984. 6, 7, 11, 14

[11] Y. Ephraim and D. Malah. Speech enhancement using a minimum mean-square error log-spectral amplitude estimator. *IEEE Trans. Acoust., Speech, Signal Process.*, ASSP-33(2):443–445, Apr. 1985. 7

[12] III Frost, Otis Lamont. An algorithm for linearly constrained adaptive array processing. *Proceedings of the IEEE*, 60(8):926–935, 1972. 2

[13] L. J. Griffiths and C. W. Jim. An alternative approach to linearly constrained adaptive beamforming. *IEEE Transactions on Antennas and Propagation*, 30(1):27–34, 1982. 3

[14] J. Hong, Y. Chu, S. C. Chan, Y. Zhao, Y. Zhou, and W. U. Ming. A robust RLS-based generalized sidelobe canceller using spatial regularization. *Digital Signal Processing*, 164:105273, 2025. 4

[15] Osamu Hoshuyama, Akihiko Sugiyama, and Akihiro Hirano. A robust adaptive beamformer for microphone arrays with a blocking matrix using constrained adaptive filters. *IEEE Transactions on Signal Processing*, 47(10):2677–2684, 1999. 4

[16] Carl Ingemarsson and Oscar Gustafsson. Hardware architecture for positive definite matrix inversion based on LDL decomposition and back-substitution. In *2016 50th Asilomar Conference on Signals, Systems and Computers*, pages 859–863. IEEE, 2016. 15

[17] Daniel Llamocca and Daniel N. Aloi. A reconfigurable fixed-point architecture for adaptive beamforming. In *2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 132–138. IEEE, 2016. 3, 11

[18] Iain A. McCowan and Hervé Bourlard. Microphone array post-filter based on noise field coherence. *IEEE Transactions on Speech and Audio Processing*, 11(6):709–716, 2003. 2, 5

[19] Tim Sainburg and Aviad Zorea. Noisereduce: Domain general noise reduction for time series signals. *arXiv preprint arXiv:2412.17851*, 2024. 5, 6, 11

[20] Suman Samui, Pratik Sahu, Indrajit Chakrabarti, and Soumya K. Ghosh. FPGA implementation of a phase-aware single-channel speech enhancement system. *Circuits, Systems, and Signal Processing*, 36(11):4688–4715, 2017. 5

[21] P. Scalart, J. Vieira Filho, and J. G. Chiquito. On speech enhancement algorithms based on MMSE estimation. In *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 1996. 7

[22] Shin-Shiang Wang, Yi-Chi Tien, Yin-Tsung Hwang, Jin-Fa Lin, and Guo-Zua Wu. MVDR based adaptive beamformer design and its fpga implementation for ultrasonic imaging. In *2016 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, pages 143–145. IEEE, 2016. 3

[23] N. Wiener. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. MIT Press, Cambridge, MA, USA, 1949. 6, 7

[24] W. Zhao, J. Q. Lin, S. C. Chan, and H. K.-H. So. A division-free and variable-regularized LMS-based generalized sidelobe canceller for adaptive beamforming and its efficient hardware realization. *IEEE Access*, 6:64470–64485, 2018. 4